

Chapter-3

Resource Monitoring and Management

System and Network Administration

- Today
 - Managing (lots of) desktops
- Loading, updating, configuring
 - Managing servers
 - Managing services

Initial OS and App Installation

- Automation solves many problems
 - Saves time/money; reduces mistakes; ensures uniformity
 - Cloning (ghosting, disk imaging) sometimes an option
- Full automation much better than partial
- Partial automation better than none
 - Needs to be well-documented for consistency

Managing (lots of) Desktops

- Three main system admin tasks for workstations
 - **Initial loading of system** software and applications
 - **Updating system** software and applications
 - **Configuring** network parameters
- Need to get all three right
 - **Initial load** must be consistent across machines
 - **Updates** must be quick
 - **Network configuration** best managed centrally
- **Solution is automation** (for supported platforms)

Lots of desktops

- *You really don't want to install, configure, and*
- *update lots of machines individually*



Use your own installation

- **Don't trust the vendor's** pre-installed OS
 - Adding apps to a truly clean installation can be easier
 - Their install image can change over time
 - You'll need to re-install eventually
- **Making your re-installation** a different configuration
- You want **to be certain** that you have everything (drivers, software, etc.) to re-install
 - You may not want or need their special applications and add-ons

Updating system and apps

- Over time, people find
 - New bugs
 - New security holes
 - New applications
- Updates can (and should) be automated, too
- Example automation systems include **Linux package updaters** like yum / apt

Network Configuration

- Network config different from install
 - Values vary by location, rather than OS+apps
- Typical solution is to use DHCP
 - Eliminate time and manual error
- By sysadmin or user (assigning himself an IP address and/or hostname)
 - More secure (only authorized systems get access)
 - Can assign a particular IP to an individual host
 - Centralized control makes updates and changes easier (e.g., new DNS server)

Managing Servers

- Different from desktop? Yes!
- May serve tens, hundreds or many thousands of users
- Requires reliability and high uptime
- Requires tighter security
- Often expected to last longer
- Extra cost is amortized across users, life span



Managing Servers (cont.)

- Servers typically have
 - Different OS configurations than desktops
 - Deployment within the **data center**
 - Maintenance contracts
 - Disk backup systems
 - Better remote access



Server Hardware

- Buy server hardware for servers
 - More internal space
 - More **CPU** performance
 - High performance **I/O** (both disk and network)
 - More upgrade options
 - **Rack mountable**/optimized
- Use **vendors known** for reliability



Data Backups

- Servers are often unique with **critical data** that must be backed up
 - **Clients** are often not backed up (most data is on server)
 - Consider separate administrative network
- Provides **alternate access** during network problems
- Requires additional **NICs, cabling, switches**

Servers in the Data Center

- Servers should be located in **data centers**
- Data centers provide
 - **Proper power** (enough power, conditioned, UPS, maybe generator)
 - **Fire protection**/suppression
 - Networking
 - Sufficient **air conditioning** (climate controlled)
 - **Physical security**



Remote Administration

- Data centers are *expensive*, and thus often cramped, cold, noisy, and may be **distant** from **admin office**
- Servers should **not require physical presence** at a console
- Typical solution is a **console server**
 - Eliminate need for keyboard and screen
 - Can see booting, can send **special keystrokes**
 - Access to console server can be remote (e.g., **ssh, rdesktop**)
- Power cycling provided by remote-access power-strips
- **??Media insertion & hardware servicing are still problems**



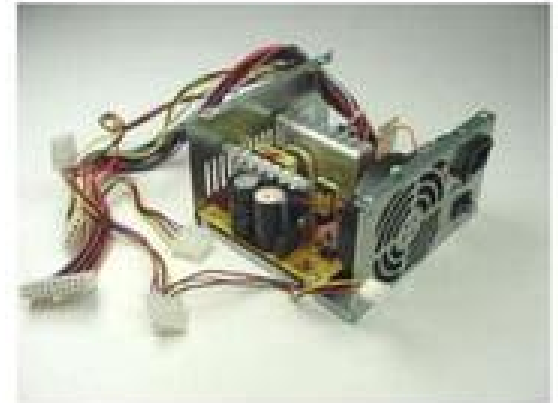
Mirrored Root Disks



- Disk drives fail!
- Often useful to consider **RAID** for data integrity
- The main system disk is often the most difficult to replace
- **Software RAID** often comes with the OS for “free”; **hardware RAID** is getting cheaper
- Two approaches for mirrored root disks:
 - **Two disks**; copy from the working disk to the clone *at regular intervals* (e.g., once a night)
 - Use **hardware or software RAID** to keep both in *sync*
- RAID disks still need to be **backed up**

Redundant Power Supplies

- Power supplies **2nd** most failure-prone part
- Ideally, servers should have redundant power supplies
 - Means the server will still operate if one power supply fails
 - Should have separate power cords
 - Should draw power from different sources (e.g., separate UPSes)



Router with dual power supplies

- This is a **Cisco 4506** switch that serves as one of the **backbone switches** for Lehigh's network.
- It has redundant power supplies, one connected to a **UPS** and one connected directly to **commercial power**



Hot-swap Components

- Redundant components should be hotswappable
 - New components can be added without **downtime**
 - Failed components can be replaced without outage
- Hot-swap components increases cost
 - But consider **cost of downtime**
- **Always check**
 - Does OS fully support hot-swapping components?
 - What parts are not hot-swappable?
 - How long/severe is the service interruption?



Managing Services

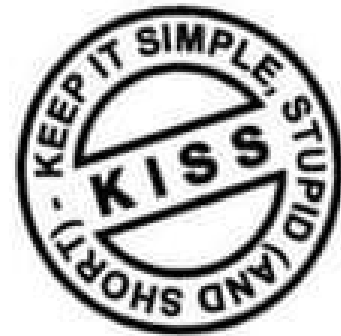
- Typical environments have many services
 - DNS, email, authentication, networking, printing
 - Remote access, license servers, DHCP, software repositories, backup services, Internet access, file service
- Providing a service means
 - Not just putting together hardware and software
 - Making service reliable
 - Scaling the service
 - Monitoring, maintaining, and supporting the service

Designing a solid service

- Get customer requirements
 - **Reason for service**
 - How service will be used
 - Features needed
 - Level of reliability required
 - Justifies budget level
 - **Define a service level agreement (SLA)**
 - specify services
 - Defines level of support provided
 - Response time commitments for various kinds of problems
 - **Estimate satisfaction** from demos or small usability trials



Designing a solid service(cont..



– Favor simplicity

- Simple systems are more reliable, easier to maintain, and less expensive
- Typically a features vs. reliability trade-off
- Take advantage of vendor relationships
- Provide recommendations for standard services
- Let multiple vendors compete for your business
- Understand where the product is going

Designing a solid service(cont..)

- **Machine independence**

- Clients should access service **using generic name**

- e.g., www, calendar, pop, imap, etc.

- **Moving services to different machines** becomes invisible to users

- Consider (right from the start) what it will take to move the service to a new machine

- **Supportive environment**

- Data center provides power, AC, security, networking

- Only rely on systems/services also found in data center (within protected environment)

- e.g., don't depend on a service from a PC in the closet

Designing a solid service(cont..)

Reliability

- Build on reliable hardware
- Exploit redundancy when available
 - Plug redundant power supply into different UPS on different circuit
- Components of service should be tightly coupled
 - Reduce single points of failure
 - e.g., all on same power circuit, network switch, etc.
 - Includes dependent services
 - e.g., authentication, authorization, DNS, etc.
- Make service as simple as possible
- Independent services on separate machines, when possible

Designing a solid service(cont..)

– Restrict access

- Customers should not need physical access to servers
- Fewer people -> more stable, more resources, more secure
- Eliminate any unnecessary services on server (security)
- Centralization and standards
- Building a service = centralizing management of service
- May be desirable to standardize the service and centralize within the organization as well
- Makes support easier, reducing training costs
- Eliminates redundant resources



Designing a solid service(cont..)

– Performance

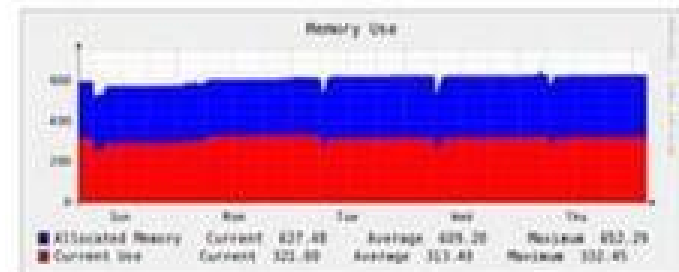
- If a service is deployed, but slow, it is **unsuccessful**
- Need to build in the ability to scale
- Can't afford to build servers for service every year
- Need to understand how the service can be split across multiple machines if needed
- Estimate capacity required for production (and get room for growth)
- First impression of user base is very difficult to correct
- When choosing hardware, consider whether service is likely
- Disk I/O, memory, or network limited

Designing a solid service

– Monitoring

- Helpdesk or front-line support must be automatically alerted to problems
- Customers that notice major problems before sysadmins are getting poor service
- Need to monitor for capacity planning as well
- Service roll-out
- First impressions
- Have all documentation available
- Helpdesk fully trained
- Use slow roll-out

Memory Use



Load Average

